

LIFT

Log Insights & Future Trajectory
기록 통찰과 미래 궤적

목차

1.개요 P.3

프로젝트 개념과 목적
기간
기술스택

2.아키텍처 P.4

3.패턴 중심 구현 요약 P.5

디자인 패턴
스니펫
API/DB 요약

4.실행 데모 P.6

영상 요약

개요

개발 목적

운동 기록을 한 번만 입력해 검증-저장-조회/분석으로 곧바로 이어지게 하고, 기록 기반 3/6/12개월 로드맵을 제공

기간

2주

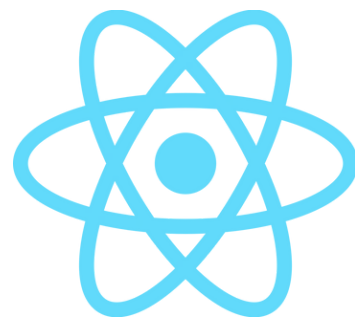
기술 스택

Backend



Flask

Frontend



React
Native



Expo

DB

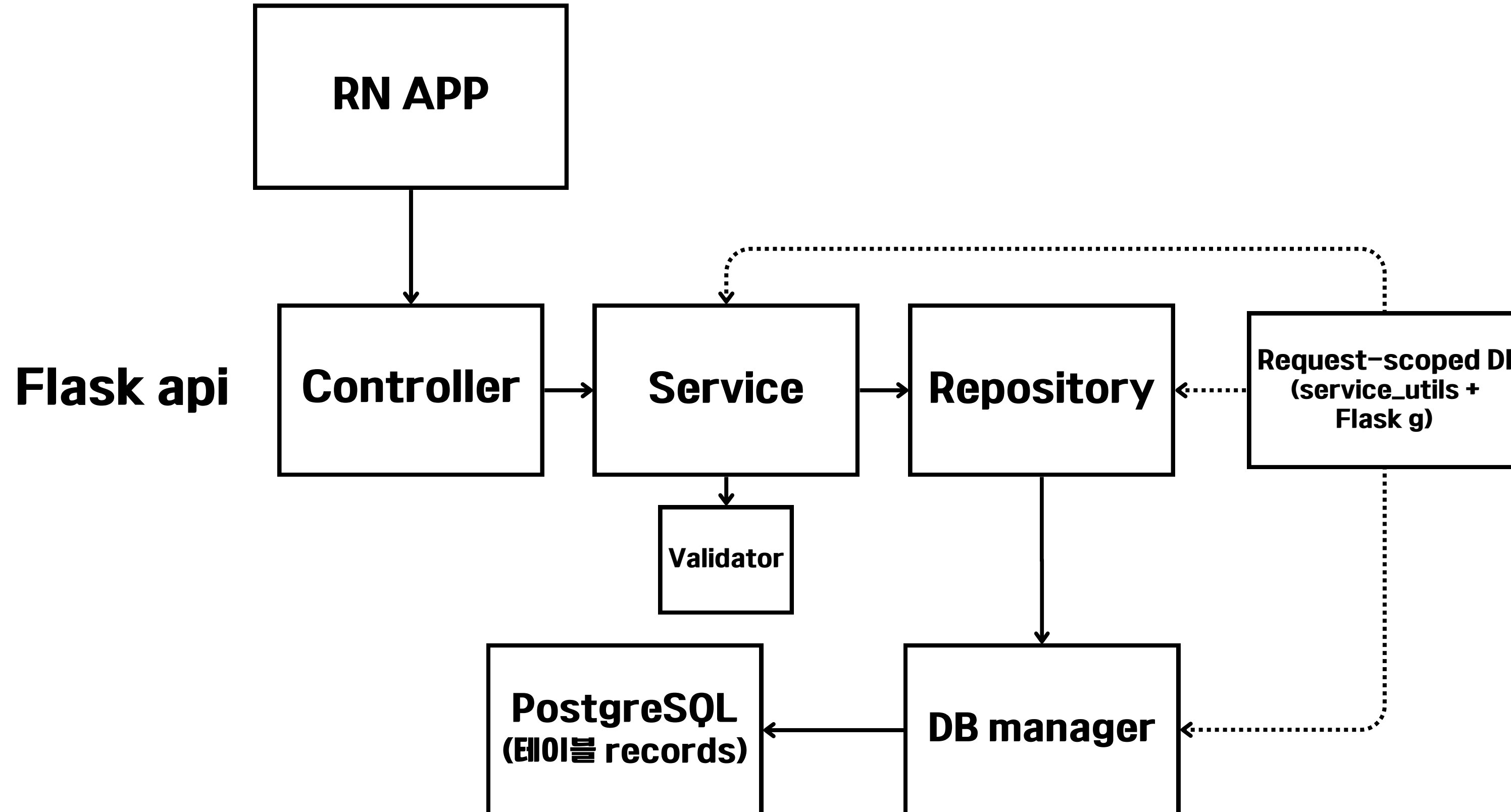


PostgreSQL

ML



아키텍처



패턴 중심 구현 요약

디자인 패턴

Layered	Controller / Validator / Service / Repository	입출력·검증·도메인·영속 분리
Validator	workout_data_validator	형식·범위 검증을 서비스에서 수행
Repository	RecordsRepository	업서트·쿼리 캡슐화, 도메인 집중
DI(Request-scoped)	service_utils + Flask g	요청마다 생성·재사용·종료 시 정리

스니펫 controller

```
@workout_bp.route('/', methods=['POST'])
def add_workout_record():
    raw_data = request.get_json()

    workout_service: WorkoutService = get_workout_service()

    new_record = workout_service.add_record(raw_data)

    return jsonify({
        "message": "Workout record saved successfully",
        "estimated_1rm_for_set": new_record.estimated_1rm
    }), 201
```

service

```
def add_record(self, raw_data: Dict[str, Any]) -> WorkoutRecord:
    validated = self.validator.validate(raw_data)
    new_record = WorkoutRecord(
        record_date=validated['record_date'],
        exercise_type=validated['exercise_type'],
        weight=validated['weight'],
        reps=validated['reps'],
        sets=validated['sets'],
    )
    inserted_count = self.repo.save(new_record)
    if inserted_count == 0:
        raise DatabaseServiceError("Failed to save workout record to database (inserted count is 0)")
    return new_record
```

repository

```
def save(self, rec: WorkoutRecord) -> int:
    return self.db.insert_records([rec])
```

validaotor

```
def validate(self, raw_data: Dict[str, Any]) -> Dict[str, Any]:
    # 1. 입력 데이터가 딕셔너리 형태인지 확인 (가분적인 안전 장치)
    if not isinstance(raw_data, dict):
        raise ValidationError("입력 데이터는 딕셔너리(JSON 객체) 형태여야 합니다.")

    # 2. 필수 필드 존재 확인
    required_fields = ['exercise_type', 'weight', 'reps', 'sets']
    if not all(field in raw_data for field in required_fields):
        missing = [field for field in required_fields if field not in raw_data]
        raise ValidationError(f"필수 필드가 누락되었습니다: {', '.join(missing)}")

    # 3. 데이터 추출 및 기본 타입 검사
    exercise_type = raw_data.get('exercise_type')
    weight = raw_data.get('weight')
    reps = raw_data.get('reps')
    sets = raw_data.get('sets')

    # 4. 운동 종목 유효성 검사 및 표준화
    if not isinstance(exercise_type, str):
        raise ValidationError("운동 종류의 데이터 타입이 유효하지 않습니다. 문자열이어야 합니다.")
    exercise_type = exercise_type.strip() # 앞뒤 공백 제거
```

Api/DB 요약

API			DB		
Path	Method	요약	컬럼	타입	비고
/workouts/	POST	기록 저장(검증→업서트→1RM for set)	record_date	DATE	INDEX(record_date, exercise_type)
/workouts/	GET	전체 조회(날짜·종목 정렬)	exercise_type	VARCHAR(50)	종목
/workouts/predict_1rm	GET	예측/추세선(3/6/12개월)	weight/reps/sets/estimated_1rm	INT	UNIQUE + ON CONFLICT

실행 데모

영상 요약

입력 → 저장 → 전체기록 조회 → 로드맵

링크: <https://youtube.com/shorts/tzSDRjL7jaA>